

Sanitizers Runtime Memory & Concurrency Cheatsheet

L0 Essence: The essence of Sanitizers is a dynamic program analysis suite integrating compiler instrumentation and runtime libraries. By using shadow memory mapping to track memory states, vector clocks to maintain happens-before partial orders, and bit-to-bit shadow mapping to trace initialization, it detects memory out-of-bounds/UAF, data races, uninitialized reads, and undefined behavior at runtime with lower overhead than traditional interpreters.

[M1.1] Memory and Concurrency Safety Pain Points

C/C++ programs are memory-unsafe by design, leading to critical safety issues:
• **Shadowing:** Static vulnerability scanners (e.g., Clang Static Analyzer) struggle with pointer aliasing analysis and suffer from path explosion, yielding high false-positive and false-negative rates.
• **Concurrency:** Traditional dynamic tools like Valgrind interpret instructions at runtime, incurring 10x-50x CPU slowdown and massive memory footprints, which prohibits their use in performance or concurrency stress testing.
• **Dynamic Detection:** Low-overhead compilation-phase instrumentation combined with high-precision runtime interception of out-of-bounds, use-after-free (UAF), and concurrent data races forms the core foundation of modern robust systems programming.

[M1.2] Compiler-Level Instrumentation Mechanism

The core architecture of LLVM Sanitizers is divided into compilation-time transformation (LLVM Pass) and runtime library support (compiler-rt):
• **Compilation:** After Clang parses source code into LLVM Intermediate Representation (IR), Sanitizers Pass traverses the IR's Basic Blocks. For every memory load/store instruction, the Pass automatically inserts check logic beforehand.
• **Runtime:** **Shadow Memory Mapping:** The instrumented code links against the compiler-rt static or dynamic library. The runtime manages memory allocations, maintains shadow memory mapping, and initializes system call interceptors.
• **Cooperative Running:** The compiler-time pass defines "where to check" while the runtime library determines "how to record and intercept," minimizing dynamic analysis overhead.

[M1.3] System Function Interception and Wrapping

Dynamic verification requires tracking the application's interactions with standard library (libc) functions:
• **LD_PRELOAD:** On Linux, the runtime library uses dynamic linker loading priorities or the LD_PRELOAD environment variable to ensure its custom wrapper functions are resolved first.
• **Weak Symbols Override:** For static linking, the runtime library overrides the weak symbol definitions of libc standard functions with its own strong symbol implementations.
• **Interceptors:** Interceptors for functions like `memcpy`, `strcpy`, and `pthread_create` inspect the shadow memory status of target buffer ranges before forwarding calls to the original libc implementations via `dlsym(RTLD_NEXT, ...)`.

[M1.4] Runtime Symbolization and Stack Unwinding

When a Sanitizer catches a safety violation, it must reconstruct and print a human-readable stack trace:
• **Stack Unwinding:** The runtime library captures the instruction pointer register (PC) at the error point. To extract call frames, it uses a fast unwinder (relying on the RBP frame pointer chain, requiring `-fno-omit-frame-pointer`) or a slow unwinder (parsing DWARF `.eh_frame`).
• **Symbolizer Engine:** The runtime library communicates via a pipe with a background daemon process, `llvm-symbolizer`. It sends the captured PC addresses to the engine, which queries ELF DWARF debug symbols.
• **Formatted Report:** The engine translates PCs to source filenames, line numbers, and function names, outputting a structured stack trace.

[M1.5] Performance and Optimization Trade-off

To balance detection accuracy and execution speed, compilation instrumentation must be paired with appropriate optimizer settings:
• **Overhead Sources:** Shadow address translation, state loading, and branching logic inserted before each load/store instruction disrupt CPU out-of-order execution, stalling pipelines.
• **Optimization Options:**
 - `-O0`: No optimization. The Pass instrument all memory reads and writes indiscriminately, causing severe performance degradation.
 - `-O1/-O2`: The compiler applies Dead Code Elimination (DCE) and redundant check merging (e.g., check once for multiple accesses to the same pointer within a loop), reducing CPU overhead to $2\times$.
 - `-fdebug-traceback`: High optimization levels may inline functions, leading to omitted call frames or minor line offsets in the printed error stack.

[M2.1] Shadow Memory Mapping Algorithm

AddressSanitizer maps the status of the application memory space onto a dedicated Shadow Memory region:
• **Linear Mapping:** Every 8 bytes of application memory are mapped to 1 byte of shadow memory, yielding an 8:1 mapping ratio.
• **Mapping Formula:**

$$\text{ShadowAddr} = (\text{AppAddr} \gg 3) + \text{Offset}$$

For Linux x86_64, the Offset constant is `0x7ffff000`.
• **Shadow Byte Encoding:**
 - `0x00`: All 8 application bytes are clean and addressable.
 - `0x01-0x07`: The first k bytes are addressable, while the remaining $8 - k$ bytes are poisoned (forbidden).
 - `0x08`: Redzones separating allocated regions.
 - `0xf0`: Freed memory blocks currently in the heap quarantine.

[M2.2] Redzones and Compiler-Time Instrumentation

To detect buffer overflows, ASan structures unaddressable buffer boundaries known as Redzones:
• **Variable Padding:** The LLVM Pass modifies the memory layout of stack and global variables during compilation. It inserts unaddressable padding (Redzones) and poisons their corresponding shadow bytes (e.g., stack left redzone is `0xf1`, middle is `0xf2`, right is `0xf3`).
• **Out-of-Bounds Assertion:** For any memory load/store instruction:
 `"e char *p = malloc(10); // Inserted by compiler: char *shadow_addr = (p >> 3) + 0x7ffff000; if (!shadow_addr != 0) ReportError(p);"`
 An index calculation shifts the pointer into a Redzone, the shadow check hits a non-zero poisoned value, triggering an immediate crash report.

[M2.3] Use-After-Free and Quarantine Protection

Use-After-Free (UAF) occurs when dangling pointers write back to reclaimed heap blocks:
• **Deallocation Hijacking:** The runtime wrapper intercepts `free(ptr)`. Instead of returning the memory to the OS, it poisons the corresponding shadow bytes to `0xf4` (freed heap).
• **Quarantine Queue:** The freed block is pushed into a First-In-First-Out (FIFO) quarantine queue (e.g., limited to 256MB). While in quarantine, the block cannot be recycled or reallocated.
• **Violation Interception:** If a dangling pointer accesses the quarantined address, ASan identifies the `0xf4` shadow byte, asserts a UAF violation, and halts. Older blocks are evicted from quarantine and reclaimed when the size limit is exceeded.

[M2.4] Heap Allocation Hijacking

ASan uses a custom allocator, `asan_allocator`, to manage physical heap memory:
• **Reconstructed Allocation:** When `malloc(size)` is called, the allocator reserves a contiguous memory region of size `Header_Size + Left_Redzone + size + Right_Redzone`, aligned to 16-byte boundaries.
• **Header Metadata:** The block header (poisoned) stores vital allocation metadata, including the allocation thread ID and a hash of the allocation stack trace.
• **Boundary Isolation:** The allocator poisons the Left and Right Redzones (e.g., `0xf4 / 0xfb`). This layout isolates buffer boundaries, ensuring even large out-of-bounds writes are blocked before corrupting neighboring allocations.

[M2.5] Stack Use-After-Return (UAR) and Use-After-Scope (UAS)

Stack variables are deallocated when their declaring function or block exits, but escaping pointers can cause bugs:
• **UAS Detection:** With `ASAN_OPTIONS=detect_stack_use_after_return=1` enabled, the LLVM Pass alters stack frames. Instead of physical stack allocation, stack variables are allocated dynamically on a heap-backed "Fake Stack." Upon return, these fake frames are poisoned to `0xf5`. Subsequent accesses by escaping pointers are intercepted.
• **UAR Detection:** When execution exits a block scope (e.g., `char tmp[10]; ...`), the compiler inserts a call to `__asan_poison_stack_memory`, poisoning the corresponding shadow bytes to `0xf8`. Accessing `tmp` after the scope block triggers an error.

[M3.1] Data Races and the Happens-Before Relation

A data race is a dangerous concurrent access pattern occurring when multiple threads read and write to the same memory without synchronization:
• **Race Definition:** Two or more threads concurrently access the same memory address, at least one access is a write, and no synchronization locks establish a relative execution order.
• **Happens-Before Relation ($\rightarrow$):** A partial order relation defining temporal dependency. If event $A \rightarrow B$, the results of A are guaranteed visible to B .
• **Race-Free Condition:** For any concurrent accesses E_1 and E_2 to the same memory, they must satisfy $E_1 \rightarrow E_2$ or $E_2 \rightarrow E_1$; otherwise, TSan flags a data race.

[M3.2] Vector Clocks State Tracking

TSan tracks happens-before relations across threads using Vector Clocks:
• **Vector Definition:** Each thread i maintains an integer array $V C_i$ of size N (total threads). $V C_j[j]$ denotes the latest logical time (epoch) of thread j known to thread i .
• **Local Tick:** On performing local reads/writes, thread i increments its own component:

$$V C_i[i] = V C_i[i] + 1$$

• **Synchronization Merge:** When thread i releases mutex L , the lock copies the clock: $V C_L = V C_i$. When thread j subsequently acquires L , it merges the clocks:

$$V C_j = \max(V C_j, V C_L)$$

• This transfers the happens-before dependency across threads.

L1 Logical Framework

Instrumentation & Interception: The compiler LLVM Pass inserts

checks at the IR level, and runtime interceptors hijack key libc syscalls, collaborating with `llvm-symbolizer` to achieve high-precision symbolization and stack unwinding.
• **Shadow Poisoning & Quarantine:** ASan establishes an 8:1 shadow memory mapping, intercepts out-of-bounds access by poisoning redzones around variables, and detects use-after-free (UAF) by delaying physical block reclamation in a quarantine queue.
• **Vector Clocks & Race Detection:** TSan tracks happens-before relations via vector clocks, matching each 8-byte application memory to 4 shadow cells (recording thread, epoch, range, write/read) to achieve lock-free concurrency race detection.
• **Bit Tracing & Conservative Sweep:** MSan uses a 1:1 bit-to-bit shadow map to verify initialization states, coupled with origin tracking to trace allocations; LSan scans the root set in a conservative GC manner on exit to detect unreachable memory and report leaks.

L2 Data Flow Topology

Compile C/C++ Code $\rightarrow$ LLVM IR $\rightarrow$ ASan/TSan/MSan Pass $\rightarrow$ Insert Instrumentation Checks $\rightarrow$ Link Runtime Library (`libclang_rt`) $\rightarrow$ Application Startup $\rightarrow$ Interceptors Hijack `malloc()/free()` $\rightarrow$ Heap Allocation $\rightarrow$ Poison Redzones (Shadow Memory set to `0xf1-0xf8`) $\rightarrow$ Application Write/Read $\rightarrow$ Compile-Time Instrumented Check: `(Addr >> 3) + Offset` $\rightarrow$ Shadow Byte Value `!= 0?` $\rightarrow$ Crash Report (ASan Global/Stack OOB) $\rightarrow$ Free Object $\rightarrow$ Move Page/Block to Quarantine (Poison Shadow Memory to `0xf4`) $\rightarrow$ Application Use-After-Free Access $\rightarrow$ Shadow Byte `== 0xf4` Check $\rightarrow$ Symbolic Stack Unwinding $\rightarrow$ Report ASan UAF $\rightarrow$ Thread A Write $\rightarrow$ Record Vector Clock & Epoch $\rightarrow$ Update TSan Shadow Cell (Thread, Epoch, Lock) $\rightarrow$ Thread B Read (No happens-before link) $\rightarrow$ Compare current Epoch with Vector Clock $\rightarrow$ Data Race detected! $\rightarrow$ Deadlock Check: Lock dependency graph circular loop $\rightarrow$ UBSan runtime check: Integer overflow check (`llvm.sadd.with.overflow`) $\rightarrow$ LSan Scan: Sweep stack/registers for pointers to allocated blocks $\rightarrow$ Report memory leak.

Trade-off Matrix: Sanitizers Overhead & Accuracy

Detection Goal & Core Algorithm: Shadow memory mapping and redzone poisoning (8:1 Shadow Map & Redzone/Quarantine) $\rightarrow$ Vector clocks and shadow cell partial order comparisons (Vector Clocks & 4x Shadow Cells) [ASan targets spatial and temporal safety (out-of-bounds, UAF) in a single-threaded context $\rightarrow$ cannot detect multi-threaded data races. TSan targets happens-before order violations in multi-threaded code $\rightarrow$ cannot detect buffer overflows or memory leaks [In development pipelines, run ASan first to ensure single-threaded memory safety, then compile with TSan to identify concurrency races].]
• **Execution Overhead Control:** 8:1 shadow memory shift and static padding (CPU slowdown 1.5x - 2.5x, Memory footprint 2x) $\rightarrow$ 4:1 compressed shadow cells and vector clock operations (CPU slowdown 5x - 10x, Memory footprint 4x - 9x) [ASan requires only simple bit shifts for address validation, making it suitable for continuous fuzzing and integration testing $\rightarrow$ but quarantine queues consume extra memory. TSan must update vector clocks and slide circular buffers for every memory access, causing a large slowdown [Use ASan for routine CI testing; isolate TSan runs for concurrent stress test suites].]
• **Bit Mapping & Origin Tracking:** 1:1 bit-to-bit shadow mapping and Origin chains (MemorySanitizer - MSan) $\rightarrow$ Conservative GC root scanning (LeakSanitizer - LSan) [MSan's 1:1 bit map provides high accuracy for uninitialized reads, tracing allocation sites via Origin ID chains $\rightarrow$ but adds 3x CPU slowdown and doubles memory usage. LSan performs root sweeps on process exit, requiring no compile-time instrumentation and adding zero runtime overhead $\rightarrow$ but can miss leaks if pointers remain in memory [Use MSan during debugging to eliminate uninitialized reads; keep LSan enabled inside ASan to audit memory leaks].]
• **Instrumentation Depth & Boundaries:** Compile-time IR-level instrumentation (LLVM IR-level Instrumentation) $\rightarrow$ Runtime interceptors (Runtime Interceptors) [IR instrumentation is highly accurate and yields zero false positives $\rightarrow$ but cannot inspect assembly or uninstrumented libraries. Runtime interceptors hijack dynamic library calls without source code $\rightarrow$ but can break if library interfaces drift [Instrument all application source code, and use suppression files to filter false positives arising from uninstrumented third-party binaries].]

[M3.3] Shadow State Compression

TSan records memory access histories using compressed shadow memory representations: `\textbf{f1}\textbf{f4}` Shadow Cells: Every 8 bytes of application memory map to 4 shadow cells (each cell is 8 bytes, totaling 32 bytes, for a 4:1 mapping ratio). `\textbf{f2}\textbf{f3}` Access Ring Buffer: These 4 cells act as a circular buffer, tracking the 4 most recent read or write events on that 8-byte application memory chunk. `\textbf{f3}\textbf{f4}`-Bit Compressed Encoding: Each cell stores: `\textbf{ThreadID}` (13 bits): The thread performing the access. `\textbf{Epoch}` (16 bits): The logical epoch of the thread during access. `\textbf{AccessRange}` (8 bits): The relative byte offset and size within the 8-byte block. `\textbf{IsWrite}` (1 bit): Flag indicating a read or write operation.

[M3.4] Lock Synchronization and State Transition

Interceptors are the bridge that transfers vector clock state during synchronization: `\textbf{f1}\textbf{f3}` Synchronization Hijacking: TSan intercepts `pthread_mutex_lock/unlock`, `pthread_cond_wait`, and atomic operations. `\textbf{f2}\textbf{f3}` Lock Bridge: `\textbf{ThreadID}` on `pthread_mutex_unlock`, the thread's vector clock `VC` thread is copied to the lock's shadow state `VC` lock, and the thread's epoch is incremented. `\textbf{ThreadID}` on `pthread_mutex_lock`, the thread's `VC` is merged with `VC` lock. `\textbf{f3}\textbf{f4}` Conflict Checking: For each memory access, TSan compares the current epoch against those stored in the 4 historical shadow cells. If the historical epoch is not less than the current thread's vector clock component, a concurrent race is detected and reported.

[M3.5] Deadlock Dynamic Detection

Static graph analysis of runtime locking relationships allows deadlocks to be predicted before they block execution: `\textbf{f1}\textbf{f3}` Lock Order Graph: TSan maintains a global directed graph representing active locking dependencies. Vertices represent locks, and edges represent lock acquisitions. `\textbf{f2}\textbf{f3}` Edge Insertion: If a thread holding lock `L1` attempts to acquire lock `L2`, TSan inserts a directed edge: `L1 → L2`. `\textbf{f3}\textbf{f4}` Cycle Analysis: Upon inserting any dependency edge, TSan runs a cycle detection algorithm. If a cycle is detected (e.g., `L1 → L2 → L1`), TSan immediately issues a deadlock warning, exposing lock inversion bugs.

[M4.1] Uninitialized Memory Read Vulnerabilities

Reading uninitialized memory is a subtle yet severe class of systems programming bugs: `\textbf{f1}\textbf{f3}` Residual Dirty Data: Recycled physical pages reallocated for new variables stack/heap space retain previous memory states. Variables declared without explicit initializers read this dirty data. `\textbf{f2}\textbf{f3}` Information Leakage: Transmitting uninitialized structures (which contain alignment padding bytes) via sockets leaks stack/heap fragments over the network. `\textbf{f3}\textbf{f4}` Control Flow Hijacking: If uninitialized data affects an `if` branch condition or is cast to a function pointer, an attacker can manipulate stack layouts to hijack control flow.

[M4.2] Shadow Memory Bit-Map Mapping

To track variable initialization at a single-bit level, MSan uses a 1:1 bit-to-bit shadow mapping: `\textbf{f1}\textbf{f3}` 1:1 Byte Mapping: Every 1 byte of application memory corresponds to 1 byte of shadow memory, yielding a 1:1 mapping ratio. `\textbf{f2}\textbf{f3}` Mapping Formula:
$$\text{ShadowAddr} = \text{AppAddr} \oplus 0x200000000000$$

`\textbf{f3}\textbf{f4}` Bit Initialization Flags: Each bit in a shadow byte indicates the initialization state of the corresponding application bit: `\textbf{0}`: The bit has been written to, indicating it is initialized. `\textbf{1}`: The bit has not been written, indicating it is uninitialized (dirty). `\textbf{f4}\textbf{f3}` Shadow Propagation: Arithmetic operations (e.g., `a = b + c`) propagate shadow bits. If `b` or `c` is uninitialized, their dirty shadow bits are propagated to the shadow representation of `a`.

[M4.3] Origin Tracking Technology

When an uninitialized read occurs, knowing only the crash location is insufficient; developers need to trace where the uninitialized memory was allocated: `\textbf{f1}\textbf{f3}` Origin Memory Mapping: Compiled with `-fsanitize-memory-track-origins`, MSan reserves extra memory. Every 4 bytes of application memory map to a 4-byte Origin space. `\textbf{f2}\textbf{f3}` Bit ID Metadata: The Origin space holds a 32-bit ID referencing a global allocation stack trace registry. `\textbf{f3}\textbf{f4}` Origin Propagation: When uninitialized memory is copied (e.g., via `memcpy`), the corresponding Origin ID is copied alongside it. When an uninitialized read eventually triggers a crash, MSan prints the full propagation chain back to the original allocation site.

[M4.4] Uninitialized Read False-Positive Suppression

To prevent excessive noise, MSan defers reporting uninitialized data until it reaches critical boundaries: `\textbf{f1}\textbf{f3}` Deferred Reporting: MSan does not report errors on simple memory loads or copies (copying uninitialized padding bytes is valid). Reports are triggered only at: `\textbf{f3}\textbf{f4}` Conditional Branches: When an uninitialized value determines an `if/while` branch transition. `\textbf{f4}\textbf{f3}` Syscall Parameters: When uninitialized data is passed to a syscall (e.g., `write`, `send`). `\textbf{f2}\textbf{f3}` Uninstrumented Code Blind Spots: MSan can only track code compiled at the LLVM IR level. Inline assembly and uninstrumented third-party libraries (e.g., precompiled dynamic libraries) do not update shadow memory, leading to potential false positives or false negatives.

[M5.1] Memory Leaks and Conservative GC Scanning

Memory leaks degrade system reliability and eventually lead to Out-Of-Memory (OOM) process termination: `\textbf{f1}\textbf{f3}` Leak Definition: Memory allocated via `malloc` loses all references to its start address before being freed, creating an unreachable memory island. `\textbf{f2}\textbf{f3}` Conservative GC Approach: LeakSanitizer (LSan) acts as the mark phase of a conservative garbage collector (GC), avoiding heavy reference-counting overhead. `\textbf{f3}\textbf{f4}` Reachability Axiom: If no pointer in the application's root set (registers, stack, globals) references the address of an allocated memory block, that block is unreachable and considered leaked.

[M5.2] LSan Root Set Conservative Sweep

LSan sweeps the application memory space using a Stop-The-World (STW) mechanism to detect leaked blocks: `\textbf{f1}\textbf{f3}` Root Set Definition: Includes global writable segments (.data and .bss), active CPU registers, and the stack frames of all running threads. `\textbf{f2}\textbf{f3}` Pointer Scanning: LSan pauses all threads (via `ptrace`) and scans the root set. It treats raw 8-byte aligned memory chunks as potential pointers. If a value falls within the `[start_addr, end_addr]` range of an allocated heap block, it marks the block as "Reachable." `\textbf{f3}\textbf{f4}` Recursive Coloring: LSan recursively scans reachable heap blocks for pointers to other blocks. At the end of the sweep, any allocated block not marked reachable is flagged as a leak, and its allocation stack trace is printed.

[M5.3] UBSan Arithmetic and Pointer Safety Checks

UndefinedBehaviorSanitizer (UBSan) targets runtime actions that violate language specifications: `\textbf{f1}\textbf{f3}` Signed Integer Overflow: In C/C++, signed overflow is undefined. The compiler replaces normal arithmetic instructions with overflow-checking intrinsics (e.g., `__int32_add_with_overflow`). If an overflow occurs at runtime, it triggers a UBSan report. `\textbf{f2}\textbf{f3}` Null Pointer Dereferences: UBSan inserts null checks `if (ptr == nullptr)` before dereferences, preventing hard segfaults and emitting informative error locations instead. `\textbf{f3}\textbf{f4}` Misaligned Access: Casting an odd memory address to an aligned pointer (e.g., `int*`) violates alignment requirements. UBSan checks `(uintptr_t)ptr % alignof(type) != 0` before accesses.

[M5.4] UBSan Bounds and Vptr Validation

Array bound checking and C++ polymorphism protection guard against out-of-bounds corruption: `\textbf{f1}\textbf{f3}` Array Bounds Checking: UBSan extracts static array bounds during compilation or dynamically calculates Variable-Length Array (VLA) sizes at runtime. For each index `i`, it asserts `if (i >= array_size)` to prevent buffer overflows. `\textbf{f2}\textbf{f3}` Vptr Validation: Before dynamic C++ virtual function calls, UBSan verifies the virtual table (vtable) pointer (vptr). It checks if the vptr points to a valid vtable layout defined in the application's metadata, preventing control flow hijacking via vtable tampering.

[M5.5] UBSan Downcasting and Control Flow Integrity (CFI)

Securing class hierarchy casting and enforcing control-flow bounds are vital for exploit mitigation: `\textbf{f1}\textbf{f3}` Downcast Verification: Casting a base class pointer to a derived class pointer (`static_cast` or `dynamic_cast`) is undefined if the underlying object is not an instance of the target derived class. UBSan queries Run-Time Type Information (RTTI) to prevent Type Confusion exploits. `\textbf{f2}\textbf{f3}` Control Flow Integrity (CFI): CFI constructs a directed Control Flow Graph (CFG) at compile time, identifying all valid targets for indirect calls (function pointers, virtual functions). At runtime, it checks target PCs before jumps. If a pointer has been overwritten and points to an unauthorized address, CFI terminates the process immediately.

[M6.1] Runtime Configuration and Suppressions

Sanitizers support fine-grained runtime control to facilitate deployment on legacy systems: `\textbf{f1}\textbf{f3}` Environment Variables: `\textbf{ASan}`: `ASAN_OPTIONS=quarantine_size_mb=64:detect_odr_violation=1` manages quarantine sizes and checks for One Definition Rule violations. `\textbf{f2}\textbf{f3}` TSan: `TSAN_OPTIONS=history_size=7` retains longer thread history records. `\textbf{f2}\textbf{f3}` Suppressions File: Used to ignore known warnings in uninstrumented third-party libraries (e.g., `race:libcrypto.so`). `\textbf{f3}\textbf{f4}` Compilation Ignorelist: Configured via `-fsanitize-ignorelist=ignorelist.txt` to bypass instrumentation for specific source files or performance-critical functions.

[M6.2] ASan Diagnostic Stack Trace Reconstruction

ASan reports follow a highly structured layout for rapid diagnostic analysis: `\textbf{f1}\textbf{f3}` Header Summary: Displays the error type (e.g., `heap-use-after-free`), the action (READ/WRITE), the thread ID, and the crash stack trace. `\textbf{f2}\textbf{f3}` Lifecycle Trace: For UAF errors, ASan outputs the stack trace where the heap block was allocated, followed by the stack trace where it was freed. `\textbf{f3}\textbf{f4}` Shadow Memory Grid: Prints the shadow bytes surrounding the offending address. The corrupted address is highlighted in brackets (e.g., `[fd]` for freed heap, `[fa]` for left redzone), providing physical evidence of the memory state.

[M6.3] TSan and MSan Integration Troubleshooting

Uninstrumented libraries and low-level asynchronous primitives are the primary sources of diagnostic issues: `\textbf{f1}\textbf{f3}` TSan False Positives: Custom locking mechanisms implemented via assembly or raw atomic instructions bypass TSan's thread synchronization detection. This results in false data race warnings. To resolve this, developers insert explicit synchronization annotations (e.g., `ANNOTATE_HAPPENS_BEFORE`). `\textbf{f2}\textbf{f3}` MSan Unpoisoning: Writing to memory via syscalls (e.g., `ioctl`) or uninstrumented libraries (e.g., `OpenSSL`) does not update shadow memory. This causes subsequent reads to trigger uninitialized memory errors. Developers must manually call `__msan_unpoison(addr, size)` after such writes.

[M6.4] Fuzzing Integration and Gold Canary Deployment

Combining Sanitizers with fuzz testing (Fuzzing) is the industry standard for vulnerability discovery: `\textbf{f1}\textbf{f3}` Fuzzing Compilation: Code is instrumented with `ASan/UBSan` and compiled as a binary target for fuzzing engines like `libFuzzer` or `AFL++`. `\textbf{f2}\textbf{f3}` Crash-Driven Exploration: The fuzzer mutates inputs to find edge cases. When a vulnerability is triggered, the Sanitizer halts the process, allowing the fuzzer to log the crashing input. `\textbf{f3}\textbf{f4}` Canary Deployments: In CI/CD pipelines, canary containers compiled with Sanitizers are run under real staging traffic to intercept memory leaks and concurrency errors before production deployment.

Zone T1: Sanitizers Compilation & Runtime CLI

`clang++ -fsanitize=address -fno-omit-frame-pointer -g main.cpp -o app`

: Compiles the application with ASan enabled. `-fno-omit-frame-pointer` ensures fast stack unwinding, and `-g` preserves DWARF debugging symbols for `llvm-symbolizer`.

`clang++ -fsanitize=thread -g -O1 main.cpp -o app` : Compiles the application

with TSan. Enabling optimization (at least `-O1`) allows the LLVM Pass to prune redundant check instructions, reducing CPU slowdown from 10x to 5x. `clang++ -fsanitize=memory`

`-fsanitize-memory-track-origins=2 -g -O2 main.cpp -o app` : Compiles the application with MSan. `track-origins=2` records the allocation stack trace and tracks the history of uninitialized memory copying. `clang++ -fsanitize=undefined main.cpp`

`-o app` : Compiles the application with UBSan to inspect undefined behaviors (overflows, misalignment, null dereferences). Errors are reported to `stderr` without halting by default.

Zone T2: Suppression Rules & Diagnostics

To ignore warnings in uninstrumented third-party libraries or suppress specific errors, define suppression files and load them at runtime: `\textbf{f1}\textbf{f3}` ASan Suppression Configuration

(`asan_supp.txt`): `\textbf{f3}` Suppress ODR violation errors in a specific shared

library `\textbf{f3}` Suppress memory leak detection for a specific legacy function `\textbf{f3}` Usage: Run the binary with: `export ASAN_OPTIONS=suppressions=asan_supp.txt:detect_leaks=1`.

`\textbf{f1}\textbf{f3}` TSan Suppression Configuration (`tsan_supp.txt`): `\textbf{f3}` Suppress data race warning for a specific global variable `\textbf{f3}` Suppress data race warning inside an uninstrumented dynamic library

`\textbf{f3}` Suppress deadlock warning for a specific mutex `\textbf{f3}` class call `\textbf{f3}` Usage: Run the binary with: `export TSAN_OPTIONS=suppressions=tsan_supp.txt`. `\textbf{f3}` Common Runtime

Option Combinations: `\textbf{f3}` `ASAN_OPTIONS=halt_on_error=0:log_path=./asan.log` : Prevents the process from exiting immediately on the first error (`halt_on_error=0`) and redirects ASan stack traces to log files. `\textbf{f3}` `MSAN_OPTIONS=poison_in_free=1` : Poisons heap blocks immediately upon release, enabling MSan to detect temporal violations in multithreaded environments.